

NAME(S):

Why use dictionaries?

Begin the following as paired-programming in class, but be sure to individually finish any problems you don't get to (hand in written answers of **0.25 engagement points**, check website for due date)

Learning goals / relevant topics

- Dictionaries (Runestone Ch. 10)

Questions and coding problems

1. Scenario: Suppose you want to store and quickly look up phone numbers by name (a phone book). Why is a dictionary better than a list for this task?

.....

.....

.....

.....

.....

.....

- Task: Define a dictionary called `phone_book` with three name-number pairs.
 - Print all names (keys).
 - Print all phone numbers (values).
 - Iterate through the dictionary and print each name and number.

2. Scenario: You are given a list of votes (names). You want to count how many votes each person received. Why is a dictionary the best structure here?

.....

.....

.....

.....

.....

- Task: Given `votes = ["Alice", "Bob", "Alice", "Charlie", "Bob", "Alice"]`, create a dictionary to count votes.
 - Print the vote counts for each person.
 - Iterate through just the keys, and then just the values.

3. Scenario: You're managing a record of students and their grades. Sometimes grades need to be updated or removed.

- Task: Define a dictionary `grades` with three students and their grades.
 - Update one student's grade (use `grades["name"] = new_grade`).
 - Remove a student from the dictionary (use `del grades["name"]`).
 - Print all key-value pairs (use `.items()` to print all pairs).

4. Scenario: You are given a list of words (possibly with repeats).

```
words = ["apple", "banana", "apple", "orange", "banana", "apple", "kiwi"]
```

- Task: Given a list, count how many times each word appears in the list.
 - You must check if a word is already in the dictionary before updating its count.
 - This is a common pattern for frequency counting and demonstrates why dictionaries are ideal for such tasks.
 - Only use: dictionary definition, printing all keys/values, iterating with `.keys()`, `.values()`, and `x, y in d.items()`, updating values, and deleting keys.
 - **Do not use any other built-in dictionary methods.**

5. Scenario: You are building a simple account creation system. You receive a list of (username, password) pairs.

- Task: For each pair, you want to create an account only if the username does not already exist in your system.
 - If the username is already taken (already in your dictionary), do not add or update the password—just skip that entry.
 - Use only: dictionary definition, checking if a key exists with `in`, adding a key-value pair, iterating with `.keys()`, `.values()`, and `x, y in d.items()`, and printing.
 - Do not use any other built-in dictionary methods.